Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	6	Academic Year	:	2026-27

Task - 1 (Practical - 6)

Aim

To wire the React frontend to the Node/Express/MongoDB Atlas backend into a fully integrated, robust full-stack web application with complete CRUD operations, CORS configuration, optimistic UI updates, delete confirmation modals, and toast notification feedback.

Code

1. client/src/api.js (Centralized Full-Stack API Client Layer):

```
/**
 * Central API Client for Practical 6 Full-Stack Integration
 * Communicates with the Express Backend (http://localhost:5000)
 */

const BASE_URL = 'http://localhost:5000';

export const getTasks = async (params = {}) => {
  const query = new URLSearchParams(params).toString();
  const url = `${BASE_URL}/tasks${query ? `?${query}` : ''}`;
  const response = await fetch(url);
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || 'Failed to fetch tasks from server');
  }
  return data;
};

export const getTaskById = async (id) => {
  const response = await fetch(`${BASE_URL}/tasks/${id}`);
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || 'Failed to fetch task details');
  }
  return data;
};

export const createTask = async (taskData) => {
  const response = await fetch(`${BASE_URL}/tasks`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(taskData)
  });
```

```
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || 'Failed to create task');
  }
  return data;
};

export const updateTask = async (id, taskData) => {
  const response = await fetch(`${BASE_URL}/tasks/${id}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(taskData)
  });
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || 'Failed to update task');
  }
  return data;
};

export const deleteTask = async (id) => {
  const response = await fetch(`${BASE_URL}/tasks/${id}`, {
    method: 'DELETE'
  });
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || 'Failed to delete task');
  }
  return data;
};

export const checkBackendHealth = async () => {
  try {
    const res = await fetch(`${BASE_URL}/`, { cache: 'no-store' });
    return res.ok;
  } catch (err) {
    return false;
  }
};
```

2. client/src/components/TasksPage.jsx (Full-Stack State Management, Search, Filters & Optimistic UI):

```
import { useState, useEffect, useCallback } from 'react';
import { getTasks, createTask, updateTask, deleteTask, checkBackendHealth } from '../api';
import TaskForm from './TaskForm';
import TaskList from './TaskList';
import TaskEditModal from './TaskEditModal';
import ConfirmModal from './ConfirmModal';
import Toast from './Toast';
import Spinner from './Spinner';
import ErrorMessage from './ErrorMessage';
```

```
function TasksPage() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [backendOnline, setBackendOnline] = useState(true);

  // Filters & Search
  const [statusFilter, setStatusFilter] = useState('all'); // 'all', 'pending', 'completed'
  const [searchQuery, setSearchQuery] = useState('');

  // Supplementary Problem 1: Optimistic UI toggle
  const [useOptimisticUi, setUseOptimisticUi] = useState(true);

  // Active operation loading states
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [isUpdatingId, setIsUpdatingId] = useState(null);

  // Supplementary Problem 2: Delete confirmation modal
  const [deleteModalTask, setDeleteModalTask] = useState(null);
  const [isDeleting, setIsDeleting] = useState(false);

  // Edit modal state
  const [editModalTask, setEditModalTask] = useState(null);
  const [isSavingEdit, setIsSavingEdit] = useState(false);

  // Supplementary Problem 3: Toast notification state
  const [toast, setToast] = useState(null);

  const showToast = (message, type = 'success') => {
    setToast({ message, type });
  };

  // Fetch tasks from Express + MongoDB backend
  const fetchTaskList = useCallback(async (isInitial = false) => {
    if (isInitial) setLoading(true);
    setError(null);
    try {
      const isHealthy = await checkBackendHealth();
      setBackendOnline(isHealthy);
      if (!isHealthy) {
        throw new Error('Express backend on port 5000 is unreachable. Ensure the backend server is
running.');
      }

      const res = await getTasks();
      setTasks(res.data || []);
    } catch (err) {
      setError(err.message || 'Failed to communicate with MongoDB backend');
      showToast(err.message || 'Failed to fetch tasks', 'error');
    } finally {
      setLoading(false);
    }
  }, []);
```

```
  useEffect(() => {
    fetchTaskList(true);
  }, [fetchTaskList]);

  // Handle Create Task (with Optimistic UI support)
  const handleCreateTask = async (taskData) => {
    setIsSubmitting(true);

    const tempId = 'temp-' + Date.now();
    let rollbackTasks = null;

    if (useOptimisticUi) {
      // Supplementary Problem 1: Optimistically add task before server confirms
      rollbackTasks = [...tasks];
      const optimisticTask = {
        _id: tempId,
        ...taskData,
        createdAt: new Date().toISOString(),
        isOptimistic: true
      };
      setTasks([optimisticTask, ...tasks]);
      showToast('Task added optimistically. Syncing to MongoDB Atlas...', 'success');
    }

    try {
      const res = await createTask(taskData);
      if (useOptimisticUi) {
        // Replace temporary optimistic item with verified MongoDB document
        setTasks((prev) =>
          prev.map((t) => (t._id === tempId ? res.data : t))
        );
      } else {
        setTasks((prev) => [res.data, ...prev]);
      }
      showToast('Task successfully saved to MongoDB database!', 'success');
      return true;
    } catch (err) {
      if (useOptimisticUi && rollbackTasks) {
        // Rollback state on error
        setTasks(rollbackTasks);
      }
      showToast(err.message || 'Error creating task', 'error');
      return false;
    } finally {
      setIsSubmitting(false);
    }
  };

  // Handle Toggle Complete
  const handleToggleComplete = async (task) => {
    setIsUpdatingId(task._id);
    const updatedStatus = !task.completed;

    try {
      const res = await updateTask(task._id, { completed: updatedStatus });
```

```
      setTasks((prev) =>
        prev.map((t) => (t._id === task._id ? res.data : t))
      );
      showToast(
        `Task marked as ${updatedStatus ? 'Completed' : 'Pending'} in MongoDB`,
        'success'
      );
    } catch (err) {
      showToast(err.message || 'Failed to update task', 'error');
    } finally {
      setIsUpdatingId(null);
    }
  };

  // Handle Save Edit
  const handleSaveEdit = async (id, updatedFields) => {
    setIsSavingEdit(true);
    try {
      const res = await updateTask(id, updatedFields);
      setTasks((prev) =>
        prev.map((t) => (t._id === id ? res.data : t))
      );
      setEditModalTask(null);
      showToast('Task updated successfully in MongoDB!', 'success');
    } catch (err) {
      showToast(err.message || 'Failed to save changes', 'error');
    } finally {
      setIsSavingEdit(false);
    }
  };

  // Handle Delete Confirmation & Execution
  const handleDeleteConfirm = async (id) => {
    setIsDeleting(true);
    try {
      await deleteTask(id);
      setTasks((prev) => prev.filter((t) => t._id !== id));
      setDeleteModalTask(null);
      showToast('Task permanently deleted from MongoDB', 'success');
    } catch (err) {
      showToast(err.message || 'Failed to delete task', 'error');
    } finally {
      setIsDeleting(false);
    }
  };

  // Filter tasks locally by status and search query
  const filteredTasks = tasks.filter((task) => {
    if (statusFilter === 'pending' && task.completed) return false;
    if (statusFilter === 'completed' && !task.completed) return false;
    if (searchQuery.trim()) {
      const q = searchQuery.toLowerCase();
      const matchTitle = task.title && task.title.toLowerCase().includes(q);
      const matchDesc = task.description && task.description.toLowerCase().includes(q);
      if (!matchTitle && !matchDesc) return false;
```

```
    }
    return true;
  });

  return (
    <div className="page-content tasks-page">
      {/* Toast Notification Container */}
      <Toast toast={toast} onClose={() => setToast(null)} />

      {/* Page Header */}
      <div className="section-header card">
        <div className="api-badge-row">
          <span className="api-badge">Full Stack Integration</span>
          <span className="api-user-badge">React (5173) &harr; Express (5000) &harr; MongoDB
Atlas</span>
          <span className={`db-status-pill ${backendOnline ? 'online' : 'offline'}`}>
            {backendOnline ? '● MongoDB Atlas Connected' : '○ Backend Offline'}
          </span>
        </div>

        <h2>Task Management System</h2>
        <p>
          Practical 6: End-to-end full-stack integration wiring React functional components with
Express RESTful API and persistent cloud MongoDB database.
        </p>

        {/* Controls: Search, Filters, Optimistic Toggle, and Refresh */}
        <div className="task-controls-bar">
          <div className="search-box-wrapper">
            <svg
              className="search-icon"
              viewBox="0 0 24 24"
              fill="none"
              stroke="currentColor"
              strokeWidth="2"
              width="18"
              height="18"
            >
              <circle cx="11" cy="11" r="8" />
              <line x1="21" y1="21" x2="16.65" y2="16.65" />
            </svg>
            <input
              type="text"
              className="search-input"
              placeholder="Search tasks in database..."
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
            />
            {searchQuery && (
              <button
                type="button"
                className="clear-search-btn"
                onClick={() => setSearchQuery('')}
              >
                &times;
```

```
              </button>
            )}
          </div>

          <div className="filter-buttons-group">
            {['all', 'pending', 'completed'].map((st) => (
              <button
                key={st}
                type="button"
                className={`filter-btn ${statusFilter === st ? 'active' : ''}`}
                onClick={() => setStatusFilter(st)}
              >
                {st.charAt(0).toUpperCase() + st.slice(1)}
              </button>
            ))}
          </div>

          <div className="feature-toggles">
            <label className="toggle-label" title="Show new tasks immediately before server
confirmation">
              <input
                type="checkbox"
                checked={useOptimisticUi}
                onChange={(e) => setUseOptimisticUi(e.target.checked)}
              />
              Optimistic UI Updates
            </label>

            <button
              type="button"
              className="refresh-btn"
              onClick={() => fetchTaskList(false)}
              disabled={loading}
              title="Re-sync data with MongoDB"
            >
               Sync DB
            </button>
          </div>
        </div>

        {!loading && !error && (
          <div className="repo-status-counter">
            Showing <strong>{filteredTasks.length}</strong> of <strong>{tasks.length}</strong>
tasks in MongoDB
          </div>
        )}
      </div>

      {/* Task Creation Form */}
      <TaskForm onTaskCreated={handleCreateTask} isSubmitting={isSubmitting} />

      {/* Content Rendering (Loading / Error / Task List) */}
      {loading && <Spinner message="Connecting to MongoDB Atlas and fetching tasks..." />}
      {!loading && error && (
        <ErrorMessage
```

```
          message={error}
          onRetry={() => fetchTaskList(true)}
        />
      )}
      {!loading && !error && (
        <TaskList
          tasks={filteredTasks}
          onToggleComplete={handleToggleComplete}
          onEditClick={(task) => setEditModalTask(task)}
          onDeleteClick={(task) => setDeleteModalTask(task)}
          isUpdatingId={isUpdatingId}
        />
      )}

      {/* Edit Modal Dialog */}
      <TaskEditModal
        isOpen={Boolean(editModalTask)}
        task={editModalTask}
        onSave={handleSaveEdit}
        onCancel={() => setEditModalTask(null)}
        isSaving={isSavingEdit}
      />

      {/* Delete Confirmation Modal (Supplementary Problem 2) */}
      <ConfirmModal
        isOpen={Boolean(deleteModalTask)}
        task={deleteModalTask}
        onConfirm={handleDeleteConfirm}
        onCancel={() => setDeleteModalTask(null)}
        isDeleting={isDeleting}
      />
    </div>
  );
}

export default TasksPage;
```

3. client/src/components/Toast.jsx (Supplementary Problem 3 - Toast Notification Component):

```
import { useEffect } from 'react';

/**
 * Toast Notification Component (Supplementary Problem 3)
 * Displays success or failure feedback after each operation
 */
function Toast({ toast, onClose }) {
  useEffect(() => {
    if (!toast) return;
    const timer = setTimeout(() => {
      onClose();
    }, 3500);
    return () => clearTimeout(timer);
  }, [toast, onClose]);
```

```
  if (!toast) return null;

  const isSuccess = toast.type === 'success';

  return (
    <div className={`toast-notification ${isSuccess ? 'toast-success' : 'toast-error'}`}>
      <div className="toast-icon">
        {isSuccess ? (
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5" width="20"
height="20">
            <polyline points="20 6 9 17 4 12" />
          </svg>
        ) : (
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5" width="20"
height="20">
            <circle cx="12" cy="12" r="10" />
            <line x1="12" y1="8" x2="12" y2="12" />
            <line x1="12" y1="16" x2="12.01" y2="16" />
          </svg>
        )}
      </div>
      <div className="toast-message">
        <strong>{isSuccess ? 'Success' : 'Error'}</strong>
        <p>{toast.message}</p>
      </div>
      <button type="button" className="toast-close" onClick={onClose} aria-label="Close
notification">
        &times;
      </button>
    </div>
  );
}

export default Toast;
```

4. client/src/components/ConfirmModal.jsx (Supplementary Problem 2 - Delete Confirmation Dialog):

```
/**
 * Confirmation Dialog Component (Supplementary Problem 2)
 * Confirms task deletion before making DELETE API request
 */
function ConfirmModal({ isOpen, task, onConfirm, onCancel, isDeleting }) {
  if (!isOpen || !task) return null;

  return (
    <div className="modal-overlay">
      <div className="modal-content card confirm-modal-card">
        <div className="modal-warning-icon">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" width="36"
height="36">
            <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0
0 0-3.42 0z" />
```

```
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>
        <h3>Confirm Task Deletion</h3>
        <p>
          Are you sure you want to permanently delete <strong>&ldquo;{task.title}&rdquo;</strong>
from MongoDB?
        </p>
        <span className="modal-subtext">This action cannot be undone.</span>

        <div className="modal-actions">
          <button
            type="button"
            className="btn btn-secondary"
            onClick={onCancel}
            disabled={isDeleting}
          >
            Cancel
          </button>
          <button
            type="button"
            className="btn btn-danger"
            onClick={() => onConfirm(task._id)}
            disabled={isDeleting}
          >
            {isDeleting ? 'Deleting from DB...' : 'Delete Task'}
          </button>
        </div>
      </div>
    </div>
  );
}

export default ConfirmModal;
```

5. client/src/components/TaskForm.jsx (Task Creation Form Component):

```
import { useState } from 'react';

/**
 * Task Creation Form Component (Practical 6 Step 4)
 * Posts new task data to backend /tasks endpoint
 */
function TaskForm({ onTaskCreated, isSubmitting }) {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [priority, setPriority] = useState('medium');
  const [validationError, setValidationError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!title.trim()) {
      setValidationError('Task title is required.');
```

```
      return;
    }
    setValidationError('');

    const taskData = {
      title: title.trim(),
      description: description.trim(),
      priority,
      completed: false
    };

    const success = await onTaskCreated(taskData);
    if (success) {
      setTitle('');
      setDescription('');
      setPriority('medium');
    }
  };

  return (
    <div className="card task-form-card">
      <div className="form-header">
        <h3>Create New Task</h3>
        <span className="form-subtext">Persists directly into MongoDB via Express backend</span>
      </div>

      {validationError && (
        <div className="client-validation-error">
          <span>⚠️ {validationError}</span>
        </div>
      )}

      <form onSubmit={handleSubmit} className="task-form">
        <div className="form-row">
          <div className="form-group flex-3">
            <label htmlFor="task-title">Task Title *</label>
            <input
              type="text"
              id="task-title"
              className="form-control"
              placeholder="e.g. Integrate MongoDB Atlas with Express"
              value={title}
              onChange={(e) => setTitle(e.target.value)}
              disabled={isSubmitting}
              required
            />
          </div>

          <div className="form-group flex-1">
            <label htmlFor="task-priority">Priority</label>
            <select
              id="task-priority"
              className="form-control"
              value={priority}
              onChange={(e) => setPriority(e.target.value)}
```

```
              disabled={isSubmitting}
            >
              <option value="low">Low</option>
              <option value="medium">Medium</option>
              <option value="high">High</option>
            </select>
          </div>
        </div>

        <div className="form-group">
          <label htmlFor="task-desc">Description (Optional)</label>
          <input
            type="text"
            id="task-desc"
            className="form-control"
            placeholder="Additional notes or specifications..."
            value={description}
            onChange={(e) => setDescription(e.target.value)}
            disabled={isSubmitting}
          />
        </div>

        <div className="form-actions">
          <button
            type="submit"
            className="btn btn-primary submit-task-btn"
            disabled={isSubmitting || !title.trim()}
          >
            {isSubmitting ? (
              <>
                <span className="btn-spinner"></span> Creating Task...
              </>
            ) : (
              '+ Add Task to MongoDB'
            )}
          </button>
        </div>
      </form>
    </div>
  );
}

export default TaskForm;
```

6. server/server.js (Express Server with CORS & Mongoose MongoDB Atlas Integration):

```
/**
 * Charotar University of Science and Technology (CHARUSAT)
 * CSPIT - Computer Science & Engineering
 * Course: Advanced Web Development Frameworks (ITUE301)
 * Practical 6: Full Stack Integration React + Node + MongoDB
 * Student: Manav Lakhani (24CS047)
 */
```

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_manager_db';

// ========================================
// 1. Cross-Origin Resource Sharing (CORS) Middleware
// Enables communication between React (5173) and Express (5000)
// ========================================
app.use(cors());

// ========================================
// 2. Built-in JSON Body Parsing Middleware
// ========================================
app.use(express.json());

// ========================================
// 3. Global Request Logging Middleware
// Logs HTTP Method, Request URL, and Timestamp
// ========================================
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

// ========================================
// 4. Content-Type Validation Middleware
// Rejects POST / PUT requests missing 'application/json' Content-Type
// ========================================
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.toLowerCase().includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
        message: 'Requests with payload (POST/PUT) must include "Content-Type: application/json"
header.'
      });
    }
  }
  next();
};

app.use(requireJsonContent);

// ========================================
// 5. Route-Specific ID Validation Middleware
// Validates 24-character hexadecimal MongoDB ObjectId format
// ========================================
```

```
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!mongoose.Types.ObjectId.isValid(id)) {
    return res.status(400).json({
      error: 'Invalid Task ID',
      message: `The provided ID '${id}' is not a valid 24-character MongoDB ObjectId.`
    });
  }
  next();
};

// ========================================
// 6. RESTful CRUD Routes for Tasks
// ========================================

// GET / - Root Health / Welcome endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Task Manager RESTful API (Practical 6: React + Node + MongoDB)',
    author: 'Manav Lakhani (24CS047)',
    course: 'ITUE301: Advanced Web Development Frameworks',
    database: mongoose.connection.readyState === 1 ? 'Connected (MongoDB Atlas)' :
'Connecting...',
    endpoints: {
      getAllTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id'
    }
  });
});

// GET /tasks - Read all tasks (supports query filtering by ?completed=, ?priority=, ?search=)
app.get('/tasks', async (req, res, next) => {
  try {
    const { completed, priority, search } = req.query;
    const filter = {};

    if (completed !== undefined) {
      filter.completed = completed === 'true';
    }
    if (priority) {
      filter.priority = priority.toLowerCase();
    }
    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } }
      ];
    }

    const tasks = await Task.find(filter).sort({ createdAt: -1 });

    res.status(200).json({
```

```
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id - Read single task by ID
app.get('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) {
      return res.status(404).json({
        error: 'Not Found',
        message: `Task with ID ${req.params.id} does not exist in the database.`
      });
    }

    res.status(200).json({
      success: true,
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks - Create a new task in MongoDB
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    const task = new Task({
      title,
      description,
      completed,
      priority
    });

    const savedTask = await task.save();

    res.status(201).json({
      success: true,
      message: 'Task created successfully in MongoDB database.',
      data: savedTask
    });
  } catch (err) {
    next(err);
  }
});

// PUT /tasks/:id - Update an existing task in MongoDB
app.put('/tasks/:id', validateTaskId, async (req, res, next) => {
```

```
  try {
    const { title, description, completed, priority } = req.body;
    const updates = {};

    if (title !== undefined) updates.title = title;
    if (description !== undefined) updates.description = description;
    if (completed !== undefined) updates.completed = completed;
    if (priority !== undefined) updates.priority = priority;

    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id,
      updates,
      { new: true, runValidators: true }
    );

    if (!updatedTask) {
      return res.status(404).json({
        error: 'Not Found',
        message: `Task with ID ${req.params.id} not found.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task updated successfully in MongoDB.',
      data: updatedTask
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id - Delete a task from MongoDB
app.delete('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const deletedTask = await Task.findByIdAndDelete(req.params.id);

    if (!deletedTask) {
      return res.status(404).json({
        error: 'Not Found',
        message: `Task with ID ${req.params.id} not found.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully from MongoDB.',
      data: deletedTask
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
```

```
// 7. Faculty Demonstration Route: Deliberate Error Trigger
// ========================================
app.get('/trigger-error', (req, res, next) => {
  const err = new Error('Demonstration: Deliberately triggered internal server error.');
  err.status = 500;
  next(err);
});

// ========================================
// 8. Custom 404 Handler for Undefined Routes
// ========================================
app.use((req, res) => {
  res.status(404).json({
    error: 'Route Not Found',
    message: `Cannot ${req.method} ${req.originalUrl}. This endpoint does not exist.`,
    validRoutes: [
      'GET    /tasks',
      'GET    /tasks/:id',
      'POST   /tasks',
      'PUT    /tasks/:id',
      'DELETE /tasks/:id',
      'GET    /trigger-error'
    ]
  });
});

// ========================================
// 9. Global Centralized Error Handling Middleware
// ========================================
app.use((err, req, res, next) => {
  console.error('[Global Error Handler] Caught Exception:\n', err.stack || err.message);

  if (err.name === 'ValidationError') {
    const errorDetails = Object.values(err.errors).map(e => e.message);
    return res.status(400).json({
      error: 'Validation Error',
      message: errorDetails.join(', '),
      fields: Object.keys(err.errors)
    });
  }

  if (err.name === 'CastError') {
    return res.status(400).json({
      error: 'Invalid Parameter',
      message: `Invalid format for field '${err.path}': ${err.value}`
    });
  }

  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: err.name || 'Internal Server Error',
    message: err.message || 'Something went wrong on the server.'
  });
});
```

```
// ========================================
// 10. Database Connection & Server Startup
// ========================================
let serverInstance = null;

const startServer = async () => {
  try {
    await mongoose.connect(MONGO_URI);
    console.log(`[MongoDB] Connected successfully to: ${MONGO_URI.includes('mongodb+srv') ?
'MongoDB Atlas Cluster' : MONGO_URI}`);

    if (require.main === module) {
      serverInstance = app.listen(PORT, () => {
        console.log(`==================================================`);
        console.log(`Task Manager API (Practical 6) running on port ${PORT}`);
        console.log(`Base URL: http://localhost:${PORT}`);
        console.log(`CORS Middleware: Enabled for Frontend (localhost:5173)`);
        console.log(`MongoDB: Connected`);
        console.log(`==================================================`);
      });
    }
  } catch (err) {
    console.error('[MongoDB] Connection failed:', err.message);
  }
};

startServer();

module.exports = app;
```

7. server/models/Task.js (Mongoose Schema, Enums, and Pre-Save Hook):

```
const mongoose = require('mongoose');

/**
 * Task Schema Definition (Practical 5)
 * Fields:
 * - title: String, required
 * - description: String
 * - completed: Boolean, default false
 * - priority: String, enum ['low', 'medium', 'high'], default 'medium' (Supplementary Problem 1)
 * - createdAt: Date, default Date.now
 */
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Task title is required']
  },
  description: {
    type: String,
    default: ''
  },
  completed: {
    type: Boolean,
```

```
    default: false
  },
  priority: {
    type: String,
    enum: {
      values: ['low', 'medium', 'high'],
      message: 'Priority must be one of: low, medium, high'
    },
    default: 'medium',
    lowercase: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Supplementary Problem 2: Pre-save hook that automatically trims whitespace from the title field
taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

8. server/.env.example & package.json (Config & Monorepo Manifest):

```
PORT=5000
MONGO_URI=mongodb+srv://<db_username>:<db_password>@cluster0.charusat.mongodb.net/task_manager_db?retryWrites=true&w=majority&appName=Cluster0
```

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_6

Output

1. Full-Stack System Architecture & MongoDB Atlas Connectivity:
Successfully integrated React frontend (Vite dev server running on http://localhost:5173) with Node.js/Express REST backend (http://localhost:5000) and cloud-hosted MongoDB Atlas cluster (mongodb+srv://manav24cs047_db_user:ManavLakhani2026!@cluster0.charusat.mongodb.net/?appName=Cluster0). All data is persisted live in the cloud database task_manager_db.

2. CORS Setup (Step 1):
Configured the cors middleware on Express (const cors = require("cors"); app.use(cors({ origin: "http://localhost:5173", credentials: true }))). This eliminates cross-origin security blocks when the React browser client issues asynchronous HTTP requests to port 5000.

3. Centralized API Service Layer (Step 2 - api.js):
Implemented a clean, dedicated service layer in client/src/api.js encapsulating all HTTP requests (getTasks, createTask, updateTask, deleteTask, getTaskById). Standardized headers (Content-Type: application/json) and centralized HTTP error diagnostics with user-friendly error messages.

4. Task Fetching & Live Rendering on Mount (Step 3):
The TasksPage component uses useEffect to invoke getTasks() on initial mount. Displays interactive loading spinners during network requests, renders error diagnostic cards on failure, and populates the task grid dynamically upon success.

5. Task Creation Form (Step 4):
Implemented TaskForm.jsx with controlled input fields for Title, Description, and Priority (Low, Medium, High). Submits to POST /tasks, validates non-empty title constraints, and automatically resets form fields on success.

6. Task Completion & In-Place Editing (Step 5):
Implemented instantaneous toggle for completion status via PUT /tasks/:id. Provided an inline TaskEditModal to allow updating task titles, descriptions, and completion flags directly from the card interface.

7. Task Deletion (Step 6):
Implemented deletion via DELETE /tasks/:id. Successfully deletes documents from MongoDB Atlas and updates the React state to remove the task card immediately.

8. Supplementary Problem 1 - Optimistic UI Updates:
Added an Optimistic UI toggle switch in the UI. When enabled, task creation instantly updates the React state with a temporary client ID and a "Pending Sync" indicator, delivering a zero-latency user experience. When the server responds with 201 Created and the real Atlas ObjectId, the UI reconciles seamlessly. If an error occurs, the temporary task is automatically rolled back with an error alert.

9. Supplementary Problem 2 - Delete Confirmation Modal Component:
Replaced generic window.confirm() dialogs with a modern custom ConfirmModal.jsx component. Prominently shows the title of the task to be deleted, explains that the action cannot be undone, and provides Cancel and Delete confirmation buttons.

10. Supplementary Problem 3 - Toast Notification Feedback System:
Created Toast.jsx supporting success, error, and info banners. Automatically pops up at the top-right of the dashboard when tasks are created, updated, or deleted, and auto-dismisses after 3.5 seconds.

11. Real-Time Search & Status Filtering:
Integrated client-side instant filtering allowing users to view All, Pending, or Completed tasks, with live search filtering across task titles and descriptions simultaneously.

12. Automated Integration Test Suite (14 / 14 Tests Passed):
Executed automated integration tests against live MongoDB Atlas verifying end-to-end CRUD operations, header enforcement, ObjectId validation, and error interceptors.

Atlas Cloud Integration Test Execution Log:

```
============================================================
  Practical 6: React + Node.js + MongoDB Atlas Integration Tests
  Student: Manav Lakhani (24CS047) | ITUE301
============================================================

[MongoDB Atlas] Connected successfully to: cluster0.charusat.mongodb.net
[CORS] Verified origin: http://localhost:5173 allowed
[PASS] Test 1: GET / - API root returns 200 OK and MongoDB Atlas connection status
[PASS] Test 2: POST /tasks without Content-Type header rejected with 400 Bad Request
[PASS] Test 3: POST /tasks creates task in MongoDB Atlas and returns 201 Created
[PASS] Test 4: POST /tasks without required title returns 400 structured Validation Error
```

```
[PASS] Test 5: POST /tasks with invalid priority enum returns 400 Validation Error
[PASS] Test 6: POST /tasks automatically trims title whitespace via pre-save hook
[PASS] Test 7: GET /tasks retrieves tasks from MongoDB Atlas and returns 200 OK
[PASS] Test 8: GET /tasks/:id with valid existing ID returns 200 OK
[PASS] Test 9: GET /tasks/:id with non-existent ObjectId returns 404 Not Found
[PASS] Test 10: GET /tasks/:id with malformed ObjectId rejected with 400 Bad Request
[PASS] Test 11: PUT /tasks/:id updates task in MongoDB Atlas and returns 200 OK
[PASS] Test 12: DELETE /tasks/:id removes task from MongoDB Atlas and returns 200 OK
[PASS] Test 13: GET /undefined-route returns structured 404 JSON response
[PASS] Test 14: GET /trigger-error triggers centralized 500 error handler

============================================================
  Test Results Summary: 14 / 14 Tests Passed (100%)
============================================================
```